

Cours 10 — Dictionnaires · Valeurs et références

Semaine 10 ■ Manuel Hachette ch. 4 (§ 4.5, 4.6) ■ 2 × 1 h

Objectifs

- Créer et manipuler un dictionnaire (clé → valeur)
- Choisir entre tuple, liste et dictionnaire
- Comprendre l'aliasing : deux noms pour un même tableau

Partie 1 — Les dictionnaires (cours A)

1.1 Associer des clés à des valeurs

Dans une liste, on accède par **position**. Dans un **dictionnaire**, on accède par **clé** :

```
age = {"Ada": 16, "Blaise": 17, "Chloé": 16}
age["Ada"]           # 16
age["Denis"] = 15    # ajout d'une entrée
age["Ada"] = 17      # modification
len(age)             # 4
del age["Blaise"]    # suppression
```

Une clé absente lève `KeyError` ; on teste avant avec `in` :

```
if "Zoé" in age:
    print(age["Zoé"])
```

1.2 Parcourir un dictionnaire

```
for prenom in age:           # parcours des clés
    print(prenom, age[prenom])

for prenom, a in age.items(): # clés et valeurs ensemble
    print(f"{prenom} a {a} ans")
```

1.3 Le bon type pour la bonne donnée

Donnée	Type	Exemple
structure fixe	tuple	point (3, 5)
collection ordonnée évolutive	liste	notes de l'année
association clé → valeur	dictionnaire	fiche {"nom": ...}

Un dictionnaire décrit très bien une **fiche** : c'est ce que nous utiliserons pour les données en tables (semaine 18).

Partie 2 — Valeurs et références (cours B)

2.1 L'expérience troublante

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)           # [1, 2, 3, 4] ... a a changé !
```

`b = a` ne copie pas le tableau : les deux noms désignent le **même objet** en mémoire (un **alias**). Modifier via `b`, c'est modifier l'unique tableau.

Avec un entier, pas de surprise : `b = a` puis `b = b + 1` crée une **nouvelle** valeur ; `a` ne bouge pas. La différence : les listes et dictionnaires sont **mutables**, on peut modifier l'objet partagé ; les entiers, chaînes et tuples sont immuables.

2.2 Copier pour de vrai

```
b = list(a)           # copie (de surface) : deux tableaux distincts
b = a[:]              # variante avec une tranche complète
```

Une fonction qui reçoit une liste reçoit une **référence** : si elle modifie la liste (via `append` ou `t[i] = ...`), l'appelant voit la modification. C'est parfois voulu (trier sur place), parfois un bug. Règle du cours : une fonction **annonce** dans sa docstring si elle modifie son argument.

2.3 Le test d'égalité

`a == b` compare les **contenus** ; `a is b` teste si c'est le **même objet**. Deux copies sont `==` mais pas `is`.

À retenir

- Dictionnaire : accès, ajout, suppression par **clé** ; `in` teste les clés ; `.items()` pour parcourir clé/valeur.
- Fiche → dictionnaire ; séquence → liste ; structure figée → tuple.
- `b = a` sur une liste crée un **alias**, pas une copie ; copie : `list(a)`.
- Une fonction peut modifier la liste qu'on lui passe — le dire dans la docstring.